

Module – Lambda Expressions and Method References

Introduction to Lambdas

Key Points

- Lambda expressions are **short-hand functions** that implement functional interfaces.
- They provide a concise syntax for passing behavior as data.
- **Syntax pattern:** (parameters) -> expression or block.
- Enable inline function definitions without creating separate classes.

Code Snippet

```
Runnable task = () -> System.out.println("Running task");  
task.run();
```

Exercise 01 – LambdaIntro.java

Create a lambda that prints “Hello Java” using `Runnable`.

Syntax Forms

Key Points

- Lambdas can have different forms based on parameter and return types.
- Single statement → no braces needed.
- Multiple statements → use braces and explicit return if needed.

Code Snippet

```
Consumer<String> show = s -> System.out.println(s);  
BiFunction<Integer, Integer, Integer> add = (a, b) -> a + b;  
Predicate<Integer> isEven = n -> { return n % 2 == 0; };
```

Exercise 02 – LambdaSyntax.java

Write three lambdas: one with no parameter, one with a single parameter, and one returning a value.

Variable Scope and Effectively Final

Key Points

- Lambdas can access **final or effectively final** variables from enclosing scope.
- You cannot modify captured variables inside a lambda.
- Local variables are copied by value (not by reference).

Code Snippet

```
int factor = 10;
Function<Integer, Integer> multiply = x -> x * factor;
// factor++; // compile-time error
System.out.println(multiply.apply(5)); // 50
```

Exercise 03 – LambdaScope.java

Try modifying a local variable captured by a lambda and observe the compiler error.

Functional Interface Targets

Key Points

- Lambdas can only target interfaces with a single abstract method.
- **Examples:** Runnable, Callable, Comparator, Predicate, Function, Supplier, Consumer.
- The interface method signature determines lambda parameters and return type.

Code Snippet

```
Comparator<String> comp = (s1, s2) -> s1.length() - s2.length();  
System.out.println(comp.compare("apple", "kiwi"));
```

Exercise 04 – ComparatorLambda.java

Implement a `Comparator` lambda that sorts strings by their last character.

Method References

Key Points

- Method references provide a shorter syntax for calling existing methods.
- Types:

1. `Class::staticMethod`
2. `object::instanceMethod`
3. `Class::instanceMethod (on parameter)`
4. `Class::new (for constructor reference).`

Code Snippet

```
List<String> names = List.of("Ravi", "Meena", "Ankit");  
names.forEach(System.out::println);
```

```
Function<String, Integer> len = String::length;  
Supplier<List<String>> creator = ArrayList::new;
```

Exercise 05 – MethodReferenceDemo.java

Rewrite a lambda expression using an equivalent method reference and verify output.

Constructor References

Key Points

- Constructor references use `ClassName::new` syntax.
- Automatically match functional interfaces whose method creates objects.
- Useful with streams and factory-like methods.

Code Snippet

```
Supplier<List<String>> listCreator = ArrayList::new;  
List<String> list = listCreator.get();
```

Exercise 06 – ConstructorRef.java

Use a constructor reference to create a new `StringBuilder` instance and append text to it.

Chaining and Composition

Key Points

- Lambdas can be chained using default methods like `andThen()`, `compose()`, and `or()`.
- Encourages reusable function pipelines.
- Common for `Function`, `Predicate`, and `Consumer`.

Code Snippet

```
Function<Integer, Integer> doubleIt = x -> x * 2;  
Function<Integer, Integer> square = x -> x * x;  
System.out.println(doubleIt.andThen(square).apply(3)); // 36
```

Exercise 07 – LambdaChain.java

Create two functions and combine them using `andThen()` and `compose()`.

Certification Tips

Key Points

- Lambdas capture variables that are effectively final.
- Method references are syntactic shortcuts for simple lambdas.
- Lambdas cannot declare checked exceptions unless the functional interface allows them.
- Functional interfaces from `java.util.function` are common exam topics.

Code Snippet

```
Predicate<String> empty = String::isEmpty;  
System.out.println(empty.test("")); // true
```

Exercise 08 – CertificationLambda.java

Create a lambda and a method reference for the same operation, then compare readability and syntax rules.

Module – Stream Creation and Pipeline Concepts

What Is a Stream

Key Points

- A **Stream** is a sequence of elements supporting **functional-style operations**.
- Introduced in Java 8 as part of the `java.util.stream` package.
- Streams process data **declaratively** without modifying the source.
- They enable efficient operations like filtering, mapping, and reduction.

Code Snippet

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);
nums.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .forEach(System.out::println);
```

Exercise 01 – StreamIntro.java

Create a Stream from a list of numbers and print only even numbers squared.

Stream Sources

Key Points

- Streams can be created from various data sources:
 - Collections → `list.stream()`
 - Arrays → `Arrays.stream(array)`
 - Static factories → `Stream.of(...)`
 - Infinite streams → `Stream.iterate()`, `Stream.generate()`.
- Each source defines the **origin** of data for the pipeline.

Code Snippet

```
Stream<String> s1 = Stream.of("A", "B", "C");  
Stream<Integer> s2 = Arrays.stream(new int[]{1, 2, 3}).boxed();  
Stream<Double> randoms = Stream.generate(Math::random).limit(3);
```

Exercise 02 – StreamSources.java

Create three Streams — one from a List, one from an array, and one using `Stream.iterate()`. Print all elements.

Stream Pipeline Architecture

Key Points

- A Stream pipeline consists of three parts:
 1. **Source** → data origin.
 2. **Intermediate Operations** → transformations (e.g., `filter()`, `map()`).
 3. **Terminal Operation** → produces a result (`forEach()`, `collect()`, `count()`).
 - Intermediate operations are **lazy**; they execute only when a terminal operation is called.

Code Snippet

```
long count = List.of("apple", "banana", "cherry", "apricot")
    .stream()
    .filter(s -> s.startsWith("a"))
    .map(String::toUpperCase)
    .count();
System.out.println(count);
```

Exercise 03 – StreamPipeline.java

Build a Stream pipeline to count the number of names starting with 'R' after converting them to uppercase.

Intermediate Operations

Key Points

- Intermediate operations return a new Stream and are **lazy**.
- Common operations:
 - `filter(Predicate<T>)`
 - `map(Function<T,R>)`
 - `distinct()`
 - `sorted()`
 - `limit(long)`
 - `skip(long)`
- Execution happens only when a terminal operation is invoked.

Code Snippet

```
List<String> result = List.of("Java", "Python", "C", "C++", "Go")
    .stream()
    .filter(s -> s.length() > 2)
    .sorted()
    .toList();
System.out.println(result);
```

Exercise 04 – IntermediateOps.java

Create a pipeline using `filter()`, `distinct()`, and `sorted()` to process a list of country names.

Terminal Operations

Key Points

- Terminal operations produce a **non-stream result** (value, collection, or side effect).
- Common terminal operations:
 - `forEach()`
 - `collect()`
 - `count()`
 - `reduce()`
 - `min()` and `max()`
 - `anyMatch()`, `allMatch()`, `noneMatch()`.
- Once a terminal operation is executed, the stream is **consumed** and cannot be reused.

Code Snippet

```
List<Integer> nums = List.of(3, 6, 9);  
int sum = nums.stream().reduce(0, Integer::sum);  
System.out.println("Sum: " + sum);
```

Exercise 05 – TerminalOps.java

Create a Stream of integers and find the sum, max, and average using terminal operations.

Laziness and Short-Circuiting

Key Points

- Streams are **lazy** – intermediate operations don't execute until a terminal operation runs.
- **Short-circuiting** operations stop processing early when the result is determined.
- Examples: `anyMatch()`, `findFirst()`, `limit()`.

Code Snippet

```
Stream.of("Ravi", "Rohit", "Ankit")
    .filter(s -> {
        System.out.println("Checking " + s);
        return s.startsWith("R");
    })
    .findFirst();
```

Exercise 06 – LazyEvaluation.java

Add a print statement inside a `filter()` and observe how many elements are processed for `findFirst()` vs `count()`.

Parallel Streams

Key Points

- Parallel streams split data into multiple chunks processed by different threads.
- Created using `parallelStream()` or `stream().parallel()`.
- Suitable for CPU-intensive operations, but not for I/O or small datasets.
- May require thread-safe collectors or side-effect-free operations.

Code Snippet

```
List<Integer> nums = IntStream.range(1, 10).boxed().toList();
int sum = nums.parallelStream()
               .reduce(0, Integer::sum);
System.out.println(sum);
```

Exercise 07 – ParallelDemo.java

Compare the execution time of a normal Stream vs a Parallel Stream for summing 1 to 1,000,000.

Certification Tips

Key Points

- Streams are **single-use**; once a terminal operation runs, the stream is closed.
- Intermediate operations are **lazy** and return new streams.
- Short-circuiting can improve performance.
- Avoid side effects inside stream operations.
- Know the difference between `stream()` and `parallelStream()` for exam questions.

Code Snippet

```
Stream<String> s = Stream.of("A", "B", "C");  
s.forEach(System.out::println);  
// s.count(); // IllegalStateException: stream has already been operated  
upon
```

Exercise 08 – StreamReuseError.java

Call two terminal operations on the same Stream and observe the runtime exception.

Module – Filter, Map, Reduce, and Collectors

Filter Operation

Key Points

- `filter(Predicate<T>)` selects elements that satisfy a condition.
- Produces a new stream containing only matching elements.
- Commonly used to remove unwanted data before transformations.

Code Snippet

```
List<Integer> nums = List.of(1, 2, 3, 4, 5, 6);  
nums.stream()  
    .filter(n -> n % 2 == 0)  
    .forEach(System.out::println);
```

Exercise 01 – FilterEven.java

Filter a list of integers to print only even numbers greater than 10.

Map Operation

Key Points

- `map(Function<T,R>)` transforms each element into another form.
- Used to convert objects, apply calculations, or extract fields.
- One input → one output element per mapping.

Code Snippet

```
List<String> names = List.of("Ravi", "Ankit", "Meena");
names.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

Exercise 02 – MapUppercase.java

Convert a list of lowercase city names to uppercase using `map()`.

Reduce Operation

Key Points

- `reduce()` combines stream elements into a single result.
- Often used for summation, concatenation, or aggregation.
- Has two forms:
 - `reduce(identity, accumulator)`
 - `reduce(accumulator)` (returns `Optional`).

Code Snippet

```
int sum = List.of(1, 2, 3, 4)
               .stream()
               .reduce(0, Integer::sum);
System.out.println(sum);
```

Exercise 03 – ReduceSum.java

Use `reduce()` to find the product of numbers in a list.

Collectors and collect()

Key Points

- `collect()` is a **terminal operation** that accumulates results into a collection or summary.
- The `Collectors` class provides common factory methods for accumulation.
- **Common examples:** `toList()`, `toSet()`, `joining()`, `counting()`, `summarizingInt()`.

Code Snippet

```
List<String> result = Stream.of("A", "B", "C")
                           .collect(Collectors.toList());
String joined = Stream.of("Java", "Python")
                     .collect(Collectors.joining(", "));
```

Exercise 04 – CollectorsDemo.java

Collect a list of student names into a single comma-separated string using `joining()`.

Grouping and Partitioning

Key Points

- `groupBy()` groups stream elements by a classifier function.
- **Returns a** `Map<key, List<values>>`.
- `partitioningBy()` splits data into two groups (true/false).
- Useful for reports, categorization, and analytics.

Code Snippet

```
List<String> words = List.of("apple", "bat", "ball", "car");
Map<Integer, List<String>> grouped =
    words.stream().collect(Collectors.groupingBy(String::length));
System.out.println(grouped);
```

Exercise 05 – GroupingPartition.java

Use `partitioningBy()` to divide numbers into even and odd groups.

Module – Sorting, Distinct, Limit, Skip, and Peek Operations

Sorting

Key Points

- `sorted()` arranges elements in natural order or by a comparator.
- Can be combined with mapping and filtering for custom sorting logic.
- For descending order, use `Comparator.reverseOrder()`.

Code Snippet

```
List<Integer> nums = List.of(5, 1, 3, 2);
nums.stream()
    .sorted()
    .forEach(System.out::println);
```

Exercise 06 – StreamSort.java

Sort employee names alphabetically, then in reverse order using a comparator.

Distinct

Key Points

- `distinct()` removes duplicate elements using `equals()` and `hashCode()`.
- Works best on streams of immutable or comparable data.
- Maintains encounter order for ordered streams.

Code Snippet

```
List<Integer> list = List.of(1, 2, 2, 3, 3, 3);  
list.stream().distinct().forEach(System.out::println);
```

Exercise 07 – DistinctDemo.java

Print unique country names from a list containing duplicates.

Limit and Skip

Key Points

- `limit(n)` restricts the stream to the first `n` elements.
- `skip(n)` discards the first `n` elements and processes the rest.
- Commonly used for pagination or sampling data.

Code Snippet

```
IntStream.rangeClosed(1, 10)
    .skip(3)
    .limit(4)
    .forEach(System.out::print); // prints 4567
```

Exercise 08 – LimitSkip.java

Generate numbers from 1–20. Skip the first 5 and print the next 10.

Peek Operation

Key Points

- `peek()` performs an intermediate action (like logging) on each element.
- Intended for **debugging**, not modification.
- Does not change the stream or its elements.

Code Snippet

```
List<Integer> nums = List.of(2, 4, 6);  
nums.stream()  
    .peek(n -> System.out.println("Processing: " + n))  
    .map(n -> n * 2)  
    .forEach(System.out::println);
```

Exercise 09 – PeekDebug.java

Use `peek()` to log elements before and after applying a transformation.

Certification Tips

Key Points

- `filter()`, `map()`, and `reduce()` are core transformations.
- `collect()` and `Collectors` provide flexible result aggregation.
- `groupingBy()` vs `partitioningBy()` → multiple vs two-way grouping.
- `distinct()`, `limit()`, `skip()`, and `peek()` are intermediate and lazy.
- Avoid modifying elements inside `peek()` – it is not guaranteed to run in parallel pipelines.

Code Snippet

```
Map<Boolean, List<Integer>> partitioned =  
    IntStream.rangeClosed(1, 6).boxed()  
        .collect(Collectors.partitioningBy(n -> n % 2 == 0));  
System.out.println(partitioned);
```

Exercise 10 – CertificationPractice.java

Create a Stream pipeline that filters numbers > 5, doubles them, removes duplicates, and collects even numbers into a list.

Module – Optional, Primitive Streams, and Stream Reduction (sum, average, min, max)

Introduction to Optional

Key Points

- `Optional<T>` is a container object that may or may not hold a non-null value.
- Prevents `NullPointerException` and encourages explicit null handling.
- Commonly used as a return type for Stream terminal operations like `findFirst()`, `findAny()`, and `reduce()`.
- Provides methods like `isPresent()`, `ifPresent()`, and `orElse()`.

Code Snippet

```
Optional<String> name = List.of("A", "B", "C")
                            .stream()
                            .filter(s -> s.equals("B"))
                            .findFirst();
name.ifPresent(System.out::println);
```

Exercise 01 – OptionalIntro.java

Use `findFirst()` on a Stream of names to get the first name starting with 'R'. If not found, print "No match found".

Optional API Methods

Key Points

- `isPresent()` → checks if a value exists.
- `get()` → returns the value or throws `NoSuchElementException`.
- `orElse(default)` → returns value or default if empty.
- `orElseGet(Supplier)` → lazily provides default value.
- `orElseThrow()` → throws an exception if value is missing.

Code Snippet

```
Optional<Integer> opt = Optional.ofNullable(null);  
System.out.println(opt.orElse(100)); // 100  
System.out.println(opt.orElseGet(() -> 200)); // 200
```

Exercise 02 – OptionalUsage.java

Create an `Optional<Double>` and demonstrate the difference between `orElse()` and `orElseGet()`.

Primitive Streams

Key Points

- Specialized streams for primitives:
 - `IntStream`, `LongStream`, `DoubleStream`.
- Avoid boxing/unboxing overhead and provide numeric utility methods.
- Created using `range()`, `rangeClosed()`, `of()`, or `mapToInt()` conversions.

Code Snippet

```
IntStream.rangeClosed(1, 5)
    .forEach(System.out::println);
```

Exercise 03 – PrimitiveStream.java

Generate numbers 1–10 using `IntStream.rangeClosed()` and print their squares.

Converting Between Object and Primitive Streams

Key Points

- Use `mapToInt()`, `mapToLong()`, `mapToDouble()` to convert object streams to primitive.
- Use `boxed()` to convert primitive streams back to object streams.
- Useful for performance optimization and numeric computations.

Code Snippet

```
List<Integer> nums = List.of(2, 4, 6);
int sum = nums.stream()
               .mapToInt(Integer::intValue)
               .sum();
System.out.println(sum);
```

Exercise 04 – BoxedConversion.java

Convert a `List<Integer>` to an `IntStream`, calculate the sum, and then convert it back to a `List<Integer>` using `boxed()`.

Reduction Operations (sum, average, min, max)

Key Points

- Numeric streams provide built-in reduction methods:
 - `sum()`, `average()`, `min()`, `max()`, `count()`.
- Return primitive values or `Optional` wrappers for empty streams.
- Useful for statistical aggregation.

Code Snippet

```
IntStream nums = IntStream.of(10, 20, 30);
System.out.println("Sum: " + nums.sum());
System.out.println("Average: " + IntStream.of(10, 20,
30).average().getAsDouble());
```

Exercise 05 – ReduceNumbers.java

Create a numeric stream and find sum, average, and min using the corresponding methods.

Using reduce() for Aggregation

Key Points

- The `reduce()` method can also perform aggregation beyond numeric streams.
- **Signature:** `reduce(identity, accumulator)`.
- Use for custom accumulation logic or concatenation.

Code Snippet

```
String joined = Stream.of("Java", "Python", "Go")
                      .reduce("", (a, b) -> a + " " + b);
System.out.println(joined.trim());
```

Exercise 06 – ReduceCustom.java

Use `reduce()` to concatenate all names in a list separated by commas.

Statistics with summarizing Collectors

Key Points

- Collectors provide summarizing methods for numeric data:
 - `summarizingInt()`, `summarizingLong()`, `summarizingDouble()`.
- Return a summary object containing count, sum, min, max, and average.

Code Snippet

```
List<Integer> nums = List.of(5, 10, 15);
IntSummaryStatistics stats =
    nums.stream().collect(Collectors.summarizingInt(Integer::intValue));
System.out.println(stats);
```

Exercise 07 – StatsCollector.java

Use `summarizingDouble()` to calculate average, min, and max of product prices.

Combining Optional with Stream Reductions

Key Points

- Stream reductions like `min()`, `max()`, `findFirst()`, and `findAny()` return `Optional`.
- Combine `Optional` with methods like `ifPresent()` or `orElse()` for safe access.
- Avoid using `get()` without checking presence.

Code Snippet

```
List<Integer> nums = List.of(8, 3, 5);
nums.stream().min(Integer::compareTo)
    .ifPresent(min -> System.out.println("Min: " + min));
```

Exercise 08 – OptionalReduce.java

Find the minimum value in a stream using `min()` and print it only if present.

Certification Tips

Key Points

- `Optional` prevents null access; use `orElse()` and `ifPresent()` instead of `get()`.
- Primitive streams provide efficient aggregation methods.
- `reduce()` and numeric methods (`sum`, `average`) are terminal operations.
- Collectors like `summarizingInt()` return statistical summaries.
- Watch for `Optional` return types in certification questions — calling `get()` on empty `optional` throws `NoSuchElementException`.

Code Snippet

```
OptionalInt max = IntStream.empty().max();
System.out.println(max.isPresent() ? max.getAsInt() : "No elements");
```

Exercise 09 – CertificationPractice.java

Write a program that generates random numbers using `IntStream.generate()`, filters those `>50`, and prints the average using `OptionalDouble`.

Module – Parallel Streams, Performance Considerations, and Best Practices

Introduction to Parallel Streams

Key Points

- Parallel streams split data into chunks and process them using multiple threads.
- Created using `parallelStream()` or `stream().parallel()`.
- Ideal for **CPU-intensive, independent** operations where order does not matter.
- Automatically uses the **ForkJoinPool.commonPool()** for task execution.

Code Snippet

```
List<Integer> nums = IntStream.rangeClosed(1, 10).boxed().toList();
int sum = nums.parallelStream()
               .reduce(0, Integer::sum);
System.out.println(sum);
```

Exercise 01 – ParallelIntro.java

Create a parallel stream to calculate the sum of 1 to 1,000,000 and compare performance with a sequential stream.

Creating Parallel Streams

Key Points

- Two ways to create parallel streams:
 - `collection.parallelStream()`
 - `stream().parallel()` from an existing sequential stream.
- Both result in a parallel pipeline using multiple threads.
- Use `isParallel()` to check mode.

Code Snippet

```
Stream<Integer> s1 = List.of(1, 2, 3).parallelStream();
Stream<Integer> s2 = Stream.of(4, 5, 6).parallel();
System.out.println(s1.isParallel()); // true
```

Exercise 02 – StreamMode.java

Create one sequential and one parallel stream. Verify both using `isParallel()`.

Parallel Execution Model

Key Points

- Parallel streams divide the data into sub-tasks processed by multiple threads.
- The **Fork/Join framework** merges partial results automatically.
- Performance depends on available CPU cores and task complexity.
- Tasks must be **independent, stateless, and side-effect-free**.

Code Snippet

```
IntStream.rangeClosed(1, 5)
    .parallel()
    .forEach(n -> System.out.println(Thread.currentThread().getName()
+ " → " + n));
```

Exercise 03 – ThreadPrint.java

Use a parallel stream and print the current thread name for each element to observe concurrent processing.

When to Use Parallel Streams

Key Points

- Use parallel streams when:
 - Processing large datasets (>10,000 elements).
 - Operations are CPU-bound, not I/O-bound.
 - Data source is easily splittable (like arrays, lists, or ranges).
- Avoid for small collections or when thread creation overhead outweighs benefits.

Code Snippet

```
List<Double> data = new Random().doubles(1_000_000).boxed().toList();
long start = System.currentTimeMillis();
data.parallelStream().map(Math::sqrt).toList();
System.out.println(System.currentTimeMillis() - start + " ms");
```

Exercise 04 – PerformanceCheck.java

Compare execution times for processing 1M elements sequentially vs. parallel.

Order and Non-Determinism

Key Points

- Parallel streams may not preserve encounter order.
- Use `forEachOrdered()` to maintain order at the cost of performance.
- Avoid operations that depend on processing order (like logging in parallel).

Code Snippet

```
List.of("A", "B", "C", "D")
    .parallelStream()
    .forEach(System.out::print);          // unpredictable order
List.of("A", "B", "C", "D")
    .parallelStream()
    .forEachOrdered(System.out::print); // ordered output
```

Exercise 05 – OrderTest.java

Run a parallel stream with both `forEach()` and `forEachOrdered()` to observe output differences.

Thread Safety and Side Effects

Key Points

- Avoid shared mutable state in parallel operations.
- Updating external variables (e.g., counters or collections) is unsafe.
- Use concurrent collectors like `toConcurrentMap()` or thread-safe structures if needed.
- Ensure accumulator functions in `reduce()` are **associative** and **stateless**.

Code Snippet

```
List<Integer> nums = IntStream.rangeClosed(1, 100).boxed().toList();
int sum = nums.parallelStream().reduce(0, Integer::sum); // safe
// Unsafe example:
// nums.parallelStream().forEach(n -> counter += n); // race condition
```

Exercise 06 – SafeSum.java

Implement two versions of sum using parallel streams: one with shared mutable variable (unsafe) and one with `reduce()` (safe).

Combining Parallel Streams with Collectors

Key Points

- Use concurrent collectors for thread-safe parallel reduction.
- Examples:
 - `Collectors.toConcurrentMap()`
 - `Collectors.groupingByConcurrent()`
- Avoid using non-concurrent collectors with parallel streams.

Code Snippet

```
Map<Integer, List<String>> grouped =  
    Stream.of("apple", "bat", "ball", "cat")  
        .parallel()  
        .collect(Collectors.groupingByConcurrent(String::length));  
System.out.println(grouped);
```

Exercise 07 – ConcurrentCollector.java

Group a list of words by length using `groupingByConcurrent()` in a parallel stream.

Performance Considerations

Key Points

- Parallelism does not guarantee better performance.
- Overhead from splitting, synchronization, and joining may slow down small tasks.
- Measure performance using large datasets and time measurements.
- Avoid blocking operations (I/O, locks) in parallel streams.

Code Snippet

```
long start = System.nanoTime();
IntStream.range(1, 1_000_000)
    .parallel()
    .filter(n -> n % 2 == 0)
    .count();
System.out.println((System.nanoTime() - start) / 1_000_000 + " ms");
```

Exercise 08 – SpeedCheck.java

Run equivalent sequential and parallel pipelines and compare timings. Identify threshold where parallelism becomes efficient.

Best Practices for Parallel Streams

Key Points

- Use only for CPU-bound operations on large data.
- Keep lambdas **stateless** and **side-effect-free**.
- Avoid modifying external variables or collections.
- Do not mix `parallel()` and manual thread management.
- Use built-in collectors and reduction methods instead of manual synchronization.

Code Snippet

```
List<String> words = List.of("Java", "Stream", "Parallel", "Lambda");
List<String> result = words.parallelStream()
                           .map(String::toUpperCase)
                           .toList();
System.out.println(result);
```

Exercise 09 – BestPractice.java

Refactor a parallel stream to remove side effects and make it safe for concurrent execution.

Certification Tips

Key Points

- `parallelStream()` returns a parallel stream directly from collections.
- Parallel streams use the **ForkJoinPool.commonPool()** by default.
- Parallel operations are not guaranteed to preserve order.
- Always ensure thread-safety when using mutable collectors.
- Expect exam questions comparing sequential vs. parallel results and performance behavior.

Code Snippet

```
Stream<String> data = Stream.of("x", "y", "z").parallel();  
System.out.println(data.isParallel()); // true
```

Exercise 10 – CertificationPractice.java

Create a parallel stream pipeline that maps, filters, and counts elements. Identify whether order of results is preserved or not.

Module – Functional Interfaces in java.util.function and Stream Exam Patterns

Overview of java.util.function Package

Key Points

- The `java.util.function` package defines core functional interfaces used with lambdas and streams.
- These interfaces represent single abstract method contracts (SAM).
- Common categories include:
 - Predicate (test)
 - Function (transform)
 - Consumer (action)
 - Supplier (provider)
 - Operator (mutation of same type).
- They enable reusable, generic functional logic throughout the API.

Code Snippet

```
Predicate<Integer> isEven = n -> n % 2 == 0;  
System.out.println(isEven.test(10)); // true
```

Exercise 01 – FunctionIntro.java

Create lambdas using `Predicate`, `Function`, and `Consumer` to filter, transform, and print a list of strings.

Predicate<T>

Key Points

- Represents a boolean-valued function: `boolean test(T t)`.
- Often used in filtering logic.
- Supports chaining using `and()`, `or()`, and `negate()`.

Code Snippet

```
Predicate<String> startsWithA = s -> s.startsWith("A");
Predicate<String> longName = s -> s.length() > 4;
List.of("Amit", "Ravi", "Ankit", "Dev")
    .stream()
    .filter(startsWithA.and(longName))
    .forEach(System.out::println);
```

Exercise 02 – PredicateChain.java

Filter words that start with “A” and have more than 3 letters using a chained predicate.

Function<T,R> and BiFunction<T,U,R>

Key Points

- `Function<T,R>` transforms input of type `T` into output of type `R`.
- `BiFunction<T,U,R>` takes two arguments and returns one result.
- Can be composed using `andThen()` and `compose()`.

Code Snippet

```
Function<Integer, Integer> doubleIt = n -> n * 2;  
Function<Integer, Integer> square = n -> n * n;  
System.out.println(doubleIt.andThen(square).apply(3)); // 36
```

Exercise 03 – FunctionCompose.java

Create two functions: one doubles and one squares an integer. Chain them with `compose()` and `andThen()`.

Consumer<T> and BiConsumer<T,U>

Key Points

- `Consumer<T>` performs an action on a value but returns nothing.
- Commonly used with `forEach()` or logging.
- `BiConsumer<T, U>` performs an action on two arguments.

Code Snippet

```
Consumer<String> printer = s -> System.out.println("Value: " + s);  
printer.accept("Hello");
```

Exercise 04 – ConsumerExample.java

Create a `BiConsumer<String, Integer>` that prints “name = age”.

Supplier<T>

Key Points

- `Supplier<T>` provides a value without taking any argument.
- Commonly used for lazy or on-demand value generation.
- Often used with `Stream.generate()` or `Optional.orElseGet()`.

Code Snippet

```
Supplier<Double> random = Math::random;  
System.out.println(random.get());
```

Exercise 05 – SupplierRandom.java

Use a supplier to generate 5 random numbers and print them using a stream.

Operator Interfaces

Key Points

- `UnaryOperator<T>` – operates on and returns the same type.
- `BinaryOperator<T>` – takes two operands of same type and returns a result.
- Useful in mathematical or reduction contexts.

Code Snippet

```
UnaryOperator<Integer> square = n -> n * n;  
BinaryOperator<Integer> add = Integer::sum;  
System.out.println(add.apply(10, 20)); // 30
```

Exercise 06 – OperatorMath.java

Create a `BinaryOperator<Double>` that calculates the average of two numbers.

Specialized Variants (Primitive Functional Interfaces)

Key Points

- To avoid boxing overhead, specialized versions exist for primitives:
 - `IntPredicate`, `IntFunction<R>`, `IntConsumer`, `IntSupplier`, `IntUnaryOperator`.
- Similarly for `Long` and `Double`.
- Work seamlessly with `IntStream`, `LongStream`, and `DoubleStream`.

Code Snippet

```
IntPredicate even = n -> n % 2 == 0;
IntConsumer print = System.out::println;
IntStream.range(1, 6).filter(even).forEach(print);
```

Exercise 07 – PrimitiveInterfaces.java

Use an `IntSupplier` to generate numbers and an `IntPredicate` to filter even ones before printing.

Functional Interfaces in Stream API

Key Points

- Stream methods accept functional interfaces as arguments:
 - `filter()` → `Predicate`
 - `map()` → `Function`
 - `forEach()` → `Consumer`
 - `reduce()` → `BinaryOperator`
 - `collect()` → `Collector`
- Understanding which functional interface applies to which stream operation is critical for exam questions.

Code Snippet

```
List.of(1, 2, 3, 4, 5)
    .stream()
    .filter(n -> n > 2)
    .map(n -> n * 10)
    .forEach(System.out::println);
```

Exercise 08 – StreamFunctions.java

Match each stream operation with its corresponding functional interface type.

Common Stream Exam Patterns

Key Points

- Identify **intermediate** vs **terminal** operations in a stream chain.
- Spot **lazy evaluation** — intermediate ops execute only when a terminal op is invoked.
- Remember that **streams are single-use** and **stateless**.
- Recognize **side-effect traps** in lambdas (modifying external variables).
- Be able to trace pipelines combining `map`, `filter`, `reduce`, and `collect`.

Code Snippet

```
int result = Stream.of(2, 3, 4)
    .filter(n -> n > 2)
    .mapToInt(n -> n * n)
    .reduce(0, Integer::sum);
System.out.println(result);
```

Exercise 09 – ExamPattern.java

Trace this stream: filter numbers > 3 , square them, and sum the results. Identify how many elements each stage processes.

Certification Tips

Key Points

- Know which functional interface each stream method uses.
- Remember that lambdas can't declare checked exceptions unless interface supports it.
- Be cautious of intermediate vs terminal operations and lazy evaluation.
- Avoid assuming execution order in parallel streams.
- Understand the difference between `map()` and `flatMap()` when applied to nested collections.

Code Snippet

```
List<List<Integer>> data = List.of(List.of(1,2), List.of(3,4));
data.stream()
    .flatMap(List::stream)
    .forEach(System.out::println);
```

Exercise 10 – FinalReview.java

Flatten a list of lists using `flatMap()` and apply functional interfaces (`Function`, `Consumer`) in the same pipeline.